AI PR Reviewer Assistant — Backend Master Plan (Production Ready Edition)

1. Project Overview

Project Name

AI PR Reviewer Assistant

Objective

Build an AI-powered engineering workflow automation platform that automatically reviews GitHub Pull Requests using AI.

The system should:

- receive GitHub webhook events
- fetch Pull Request changes
- analyze code changes using AI
- generate structured review suggestions
- post comments back to GitHub PR
- expose APIs for frontend dashboard
- support scalable async review processing
- prevent duplicate webhook processing
- optimize AI token usage and cost

---

2. Problem Statement

Code review is one of the most repetitive and time-consuming engineering tasks.

Common problems:

- reviewers spend too much time reading PR diffs
- small issues are often missed
- validation/security issues may slip through
- repetitive comments appear frequently
- onboarding junior developers is slower
- large PRs are difficult to review consistently
- engineering teams lose time on repeated review patterns

This project aims to automate the first layer of code review using AI.

---

3. Solution

The system automatically analyzes Pull Requests when:

- a PR is opened
- a PR is updated
- a review retry is requested

The architecture should be asynchronous.

Workflow:

```
GitHub PR Event
        ↓
GitHub Webhook
        ↓
Verify Signature
        ↓
Deduplicate Webhook
        ↓
Push Review Job To Queue
        ↓
Return 200 Immediately

Worker Process
        ↓
Fetch PR Files/Diff
        ↓
Filter Unsupported Files
        ↓
Build AI Prompt
        ↓
OpenAI Analysis
        ↓
Save Review Result
        ↓
Post Comment Back To GitHub PR
        ↓
Frontend Dashboard Displays Result
```

---

4. Technical Stack

Backend

- Node.js
- ExpressJS
- TypeScript

Database

- PostgreSQL
- Prisma ORM

AI

- OpenAI API

Queue & Background Processing

- Redis
- BullMQ

External Integrations

- GitHub Webhook API
- GitHub REST API

Observability & Logging

- Pino
- OpenTelemetry (optional)

Deployment

- Docker
- Railway / Render / VPS

---

5. Core Features (MVP)

Feature 1 — GitHub Webhook Receiver

The backend must receive webhook events from GitHub.

Supported events:

- pull_request.opened
- pull_request.synchronize
- pull_request.reopened

Responsibilities:

- validate event type
- verify GitHub signature
- deduplicate webhook
- enqueue review job
- return immediately

Feature 2 — Pull Request Fetcher

The backend fetches changed files from GitHub API.

Endpoint:

```
GET /repos/{owner}/{repo}/pulls/{pull_number}/files
```

Required data:

- filename
- patch/diff
- additions
- deletions
- file status

- blob url
- line changes

The system should:

- paginate large PRs
- filter unsupported files
- ignore generated files
- ignore binaries
- enforce token budget

Feature 3 — AI Review Engine

The backend sends filtered PR diff data to OpenAI.

AI analyzes:

- possible bugs
- validation issues
- security concerns
- race conditions
- naming improvements
- clean code suggestions
- duplicated logic
- scalability concerns
- maintainability issues

AI responses must return STRICT JSON.

Expected AI output:

```
{
  "summary": "This PR adds payment retry logic.",
  "issues": [
    {
      "severity": "HIGH",
      "type": "VALIDATION",
      "message": "Missing validation in payment controller",
      "suggestion": "Validate amount before processing payment",
      "filePath": "payment.controller.ts",
      "line": 24
    }
  ]
}
```

Feature 4 — GitHub PR Commenting

The backend automatically comments review results back to GitHub.

Supported:

- general PR review comment
- future inline review comments

Endpoints:

```
POST /repos/{owner}/{repo}/issues/{issue_number}/comments
```

Future:

```
POST /repos/{owner}/{repo}/pulls/{pull_number}/reviews
```

Feature 5 — Review Persistence

The backend stores:

- repository data
- PR metadata
- review runs
- AI outputs
- token usage
- processing status
- retry history
- prompt version
- model version

Feature 6 — REST API For Frontend

Required APIs:

```
GET /api/reviews
GET /api/reviews/:id
```

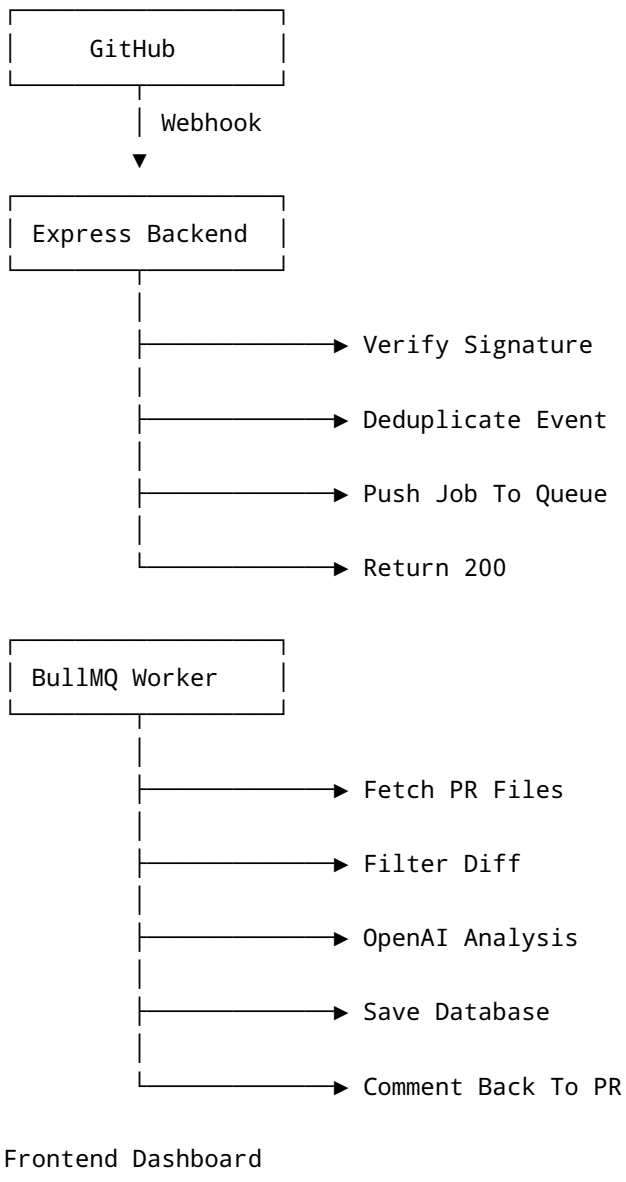
```
GET /api/pull-requests
GET /api/pull-requests/:id
```

Optional APIs:

```
POST /api/reviews/:id/retry
GET /api/stats
```

---

6. System Architecture



```
        ▲
        │ REST API
        ▼
┌──────────────────┐
│ PostgreSQL DB    │
└──────────────────┘
```

---

7. Folder Structure

```
src/
├── modules/
│   ├── webhook/
│   │   ├── webhook.controller.ts
│   │   ├── webhook.route.ts
│   │   ├── webhook.service.ts
│   │   └── webhook.validator.ts
│   │
│   ├── github/
│   │   ├── github.service.ts
│   │   ├── github.client.ts
│   │   └── github.types.ts
│   │
│   ├── ai/
│   │   ├── openai.service.ts
│   │   ├── prompt.builder.ts
│   │   ├── prompt.templates.ts
│   │   └── ai-response.parser.ts
│   │
│   ├── review/
│   │   ├── review.controller.ts
│   │   ├── review.service.ts
│   │   ├── review.repository.ts
│   │   └── review.mapper.ts
│   │
│   ├── queue/
│   │   ├── review.queue.ts
│   │   ├── review.worker.ts
│   │   └── queue.events.ts
│   │
│   └── comment/
│       ├── comment.service.ts
│       └── comment.formatter.ts
│
```

```
├── shared/
│   ├── middlewares/
│   ├── utils/
│   ├── constants/
│   ├── errors/
│   └── logger/
│
├── prisma/
│   └── client.ts
│
├── app.ts
└── server.ts
```

---

8. Database Design

Table: pull_requests

```
model PullRequest {
  id            String      @id @default(cuid())
  repoOwner     String
  repoName      String
  prNumber      Int
  prTitle       String
  author        String
  baseBranch    String
  headBranch    String
  headSha       String
  githubUrl     String
  createdAt     DateTime    @default(now())
  updatedAt     DateTime    @updatedAt

  reviews       ReviewRun[]

  @@unique([repoOwner, repoName, prNumber])
}
```

Table: review_runs

```
model ReviewRun {
  id               String   @id @default(cuid())
  pullRequestId    String

  status           ReviewStatus

  aiSummary        String?
  modelName        String
  promptVersion    String

  inputTokens      Int?
  outputTokens     Int?
  totalTokens      Int?
  estimatedCostUsd Float?

  startedAt        DateTime?
  completedAt      DateTime?

  errorMessage     String?

  createdAt        DateTime @default(now())
  updatedAt        DateTime @updatedAt

  pullRequest      PullRequest @relation(fields: [pullRequestId], references:
[id])

  comments         ReviewComment[]
}
```

Table: review_comments

```
model ReviewComment {
  id           String   @id @default(cuid())
  reviewRunId  String

  severity     SeverityLevel
  type         String
  message      String
  suggestion   String?
```

```
  filePath     String?
  line         Int?

  reviewRun    ReviewRun @relation(fields: [reviewRunId], references: [id])
}
```

Table: processed_webhooks

```
model ProcessedWebhook {
  id           String   @id @default(cuid())
  deliveryId   String   @unique
  event        String
  createdAt    DateTime @default(now())
}
```

Enums

```
enum ReviewStatus {
  RECEIVED
  QUEUED
  FETCHING_PR
  ANALYZING
  COMMENTING
  COMPLETED
  FAILED
}

enum SeverityLevel {
  LOW
  MEDIUM
  HIGH
}
```

---

9. Environment Variables

```
PORT=3000
NODE_ENV=development
```

```
DATABASE_URL=
REDIS_URL=

OPENAI_API_KEY=
OPENAI_MODEL=gpt-4.1-mini

GITHUB_TOKEN=
GITHUB_WEBHOOK_SECRET=

LOG_LEVEL=info
```

---

10. GitHub Authentication Strategy

MVP

Use GitHub Personal Access Token.

Production Recommendation

Use GitHub App instead of Personal Access Token.

Benefits:

- repository-scoped permissions
- better security
- installation-based auth
- scalable multi-user support

---

11. GitHub Webhook Setup

Step 1

Create GitHub repository.

Step 2

Go to:

```
Repository → Settings → Webhooks
```

Step 3

Add webhook URL:

```
https://your-domain.com/api/webhooks/github
```

Step 4

Select events:

```
Pull requests
```

Step 5

Add webhook secret.

---

12. API Design

Webhook API

POST /api/webhooks/github

Responsibilities:

- verify signature
- detect event type

- deduplicate event
- enqueue review job

Response:

```
{
  "success": true
}
```

Reviews API

GET /api/reviews

Returns all review runs.

GET /api/reviews/:id

Returns review detail.

POST /api/reviews/:id/retry

Retries failed review.

Stats API

GET /api/stats

Returns:

- total reviews
- failed reviews
- average processing time
- token usage
- estimated AI cost

---

13. AI Prompt Design

System Prompt

```
You are a senior backend engineer reviewing a pull request.

Focus on:
- bugs
- validation
- security
- race conditions
- scalability
- clean code
- naming
- duplicated logic

Return STRICT JSON ONLY.
```

User Prompt

```
Analyze the following pull request diff.

Return JSON response using this schema:

{
  "summary": string,
  "issues": [
    {
      "severity": "LOW" | "MEDIUM" | "HIGH",
      "type": string,
      "message": string,
      "suggestion": string,
      "filePath": string | null,
      "line": number | null
    }
  ]
}
```

---

14. Review Processing Flow

Step 1 — Receive Webhook

```
POST /api/webhooks/github
```

Step 2 — Verify Signature

Use:

```
X-Hub-Signature-256
```

Step 3 — Deduplicate Event

Use:

```
X-GitHub-Delivery
```

Skip already processed webhooks.

Step 4 — Push Queue Job

Push review job into BullMQ.

Step 5 — Worker Fetches PR Files

Call GitHub API.

Step 6 — Filter Files

Ignore:

- lock files
- generated files
- minified files
- binaries
- snapshots
- node_modules
- dist
- coverage

Step 7 — Build AI Prompt

Combine:

- PR title
- PR description
- filtered diffs

Step 8 — Send To OpenAI

Receive structured review result.

Step 9 — Save Database

Store:

- summary
- issues
- token usage
- model version
- status

Step 10 — Comment PR

Post review back to GitHub.

15. Status Flow

```
RECEIVED
↓
QUEUED
↓
FETCHING_PR
↓
ANALYZING
↓
COMMENTING
↓
COMPLETED
```

Failure path:

```
FAILED
```

16. Idempotency Strategy

GitHub may send duplicate webhooks.

The system must:

- store delivery IDs
- skip already processed events
- avoid duplicate comments
- avoid repeated OpenAI calls

17. Error Handling

Handle cases:

- invalid webhook signature
- unsupported event
- no changed files
- GitHub API failure
- OpenAI timeout

- invalid AI JSON
- database failure
- Redis connection failure
- duplicate webhook

---

18. Retry Strategy

Retry:

- OpenAI calls
- GitHub API calls
- comment posting
- queue jobs

Use:

- exponential backoff
- retry limits
- dead letter queue

---

19. Rate Limiting Strategy

Handle:

- GitHub API rate limits
- OpenAI API limits

Solutions:

- queue concurrency control
- retry with delay
- request throttling
- token budgeting

---

20. AI Cost Optimization

Strategies:

- summarize large diffs
- chunk large PRs
- skip unchanged files

- limit token usage
- cache repeated reviews
- ignore low-value files
- limit max files per review

---

21. Security Considerations

MUST HAVE

Verify GitHub Signature

Prevent fake webhook requests.

Use Environment Variables

Never expose:

- OpenAI API key
- GitHub token
- Redis credentials

Ignore Dangerous Files

Optional:

- binaries
- generated code
- huge diffs

Validate AI Responses

Never trust AI output directly.

Validate:

- schema
- types
- lengths
- allowed enums

22. Logging

Recommended logs:

```
Webhook received
Webhook deduplicated
Queue job created
Fetching PR files
Generating AI review
Saving review
Posting GitHub comment
Review completed
```

Log metadata:

- request id
- review id
- PR number
- repository
- processing time

23. Observability

Track:

- processing time
- queue size
- OpenAI latency
- failed reviews
- retry count
- token usage
- AI cost

Optional:

- Grafana
- Prometheus
- OpenTelemetry

24. Testing Strategy

Unit Tests

Test:

- services
- prompt builders
- validators
- parsers

Integration Tests

Test:

- webhook flow
- GitHub API integration
- database operations
- queue processing

E2E Tests

Test:

- full PR review lifecycle
- retry flow
- failed job recovery

---

25. Deployment Suggestions

Backend Hosting

Options:

- Railway
- Render
- VPS
- Fly.io

Database Hosting

Options:

- Neon PostgreSQL
- Supabase

Redis Hosting

Options:

- Upstash Redis
- Railway Redis

Local Webhook Testing

Use:

```
ngrok
```

Example:

```
ngrok http 3000
```

---

26. Development Roadmap

Phase 1 — MVP

Goal

Webhook flow working.

Tasks

- setup ExpressJS + TypeScript
- setup Prisma + PostgreSQL
- create webhook endpoint
- verify GitHub signature
- fetch PR files

Phase 2 — AI Review Engine

Goal

AI review working.

Tasks

- integrate OpenAI
- build prompt
- parse structured AI response
- save review result
- comment back to GitHub PR

Phase 3 — Queue & Reliability

Goal

Production-ready architecture.

Tasks

- integrate BullMQ
- add retry strategy
- implement idempotency
- improve logging
- optimize token usage

Phase 4 — Dashboard & Polish

Goal

Demo-ready platform.

Tasks

- expose frontend APIs
- build stats endpoint
- improve observability
- write README
- deploy application
- record demo

---

27. Suggested Commit Strategy

```
git commit -m "init express typescript backend"
git commit -m "setup prisma postgres and redis"
git commit -m "implement github webhook verification"
git commit -m "add webhook deduplication"
git commit -m "setup bullmq review queue"
git commit -m "implement github pull request fetcher"
git commit -m "build ai review engine"
git commit -m "parse structured ai responses"
git commit -m "implement github review comments"
git commit -m "persist review runs and comments"
git commit -m "add retry strategy and logging"
git commit -m "implement review dashboard apis"
```

---

28. README Structure Suggestion

```
# AI PR Reviewer Assistant

## Problem
...

## Solution
...
```

```
## Features
...

## System Architecture
...

## Queue Architecture
...

## Tech Stack
...

## Database Design
...

## AI Review Flow
...

## Setup
...

## Demo
...

## Reflection
...
```

---

29. Demo Flow

Demo Scenario

1. Create Pull Request
2. GitHub sends webhook
3. Backend verifies signature
4. Queue job created
5. Worker fetches PR files
6. AI analyzes code
7. Review saved to DB
8. GitHub PR receives AI comment
9. Frontend dashboard displays review result

---

30. Future Improvements

Possible future enhancements:

- inline code comments
- suggested code changes
- semantic code analysis
- repository-specific rules
- custom AI prompts
- review score system
- Slack/Discord notifications
- GitHub App multi-user support
- support GitLab/Bitbucket
- vector search for previous reviews
- caching and optimization
- multi-model AI pipeline
- AST-based code analysis

---

31. Final Goal

This project demonstrates:

- backend engineering skills
- webhook integrations
- AI workflow automation
- GitHub API integration
- queue architecture
- scalable backend design
- production engineering mindset
- observability and reliability practices
- clean architecture
- engineering productivity mindset

The final result should feel like a real internal engineering productivity platform rather than a simple CRUD application.